

Instituto Tecnológico de Colima

Ingeniería en Inteligencia Artificial

Cómputo en la Nube

Proyecto Nebula Arena — Unidad 3

Práctica 3.5 — Tabla de líderes serverless

Jarom Cortes Perez

Carlos Antonio Vogel Romero

11 de noviembre de 2026

Introducción

Al terminar cada partida, el servidor tiene que guardar los puntos de los jugadores en algún lugar, y el juego tiene que mostrar el top. Para eso no vale la pena tener una máquina encendida todo el día: la tabla se usa a ratos. La opción *serverless* cobra solo por cada petición y escala sola. En esta práctica hicimos la tabla de líderes con Lambda, API Gateway y DynamoDB, la sometimos a una prueba de carga y comparamos su costo con el de una máquina virtual.

Objetivo

Recibir el resultado de cada partida y mantener el top global sin administrar servidores, medir su comportamiento con carga y comparar el costo con una VM pequeña encendida todo el mes.

Lo que hicimos

1. La función y sus pruebas

Escribimos la función en Python 3.12 (`serverless/aws/lambda_function.py`) con la misma API que la versión de Cloud Run del material:

- `POST /score` registra el resultado de una partida.
- `GET /top` devuelve el top 10.
- `GET /jugador/{id}` devuelve las estadísticas de un jugador.

Los puntos se suman con una operación atómica de DynamoDB (`ADD`), como en el material, para que dos partidas que terminan al mismo tiempo no se pisen. Las escrituras exigen una llave (`X-Api-Key`), que guardamos en SSM Parameter Store como SecureString, fuera del código y de Terraform. La Lambda solo conoce el nombre del parámetro. Escribimos 20 pruebas y todas pasaron.

2. Desplegarla con Terraform

Con Terraform creamos 13 recursos: la tabla de DynamoDB (cobro por uso), la Lambda, su rol con permisos mínimos (solo su tabla, sus logs y su parámetro), la API de API Gateway con límite de peticiones y los logs con 7 días de retención.

```
Apply complete! Resources: 13 added, 0 changed, 0 destroyed.

Outputs:

url_tabla_lideres = "https://5d7wm0pa26.execute-api.us-east-2.amazonaws.com/"
```

Figura 1. *Final del terraform apply de la tabla de líderes: 13 recursos creados y la URL pública.*

3. Probar la URL

Desde AWS CloudShell creamos la llave y probamos la URL con los `curl` del material. Registramos 4 partidas (respuesta 201), comprobamos que sin llave se rechaza (401) y que no acepta puntos negativos (400), y consultamos el top:

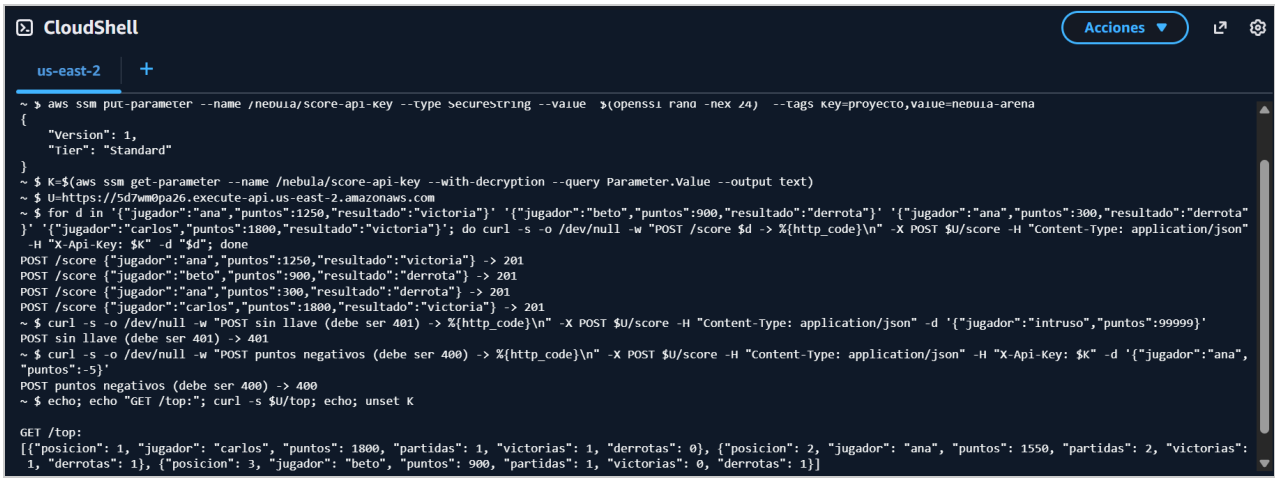


Figura 2. *Prueba de la URL pública desde AWS CloudShell con los curl del material: 4 partidas registradas (201), sin llave rechazada (401), puntos negativos rechazados (400) y el top resultante.*

También leímos la tabla desde la PC, fuera de AWS. La primera lectura tardó 149 ms y la mediana de 20 lecturas fue de 125 ms:

```
Prueba de lectura desde fuera de AWS (PC del alumno), 2026-11-09 13:54
GET /top
[{"posicion": 1, "jugador": "carlos", "puntos": 1800, "partidas": 1, "victorias": 1, "derrotas": 0}, {"posicio
n": 2, "jugador": "ana", "puntos": 1550, "partidas": 2, "victorias": 1, "derrotas": 1}, {"posicion": 3, "jugad
or": "beto", "puntos": 900, "partidas": 1, "victorias": 0, "derrotas": 1}]

GET /jugador/ana
{"jugador": "ana", "puntos": 1550, "partidas": 2, "victorias": 1, "derrotas": 1}

Tiempos de 20 lecturas de /top (ms):
149, 129, 116, 125, 132, 124, 138, 126, 132, 124, 125, 134, 121, 120, 116, 127, 129, 125, 149, 123
primera (arranque en frio o conexion nueva): 149 ms; mediana: 125 ms; p95: 149 ms
```

Figura 3. *Lecturas desde la PC, fuera de AWS: GET /top, GET /jugador/ana y los tiempos de 20 lecturas.*

4. Prueba de carga y lo que encontramos

Con Locust lanzamos desde la PC 500 usuarios durante 5 minutos: 80 % de lecturas del top y 20 % de escrituras. El resultado fue malo: de unas 440 peticiones por segundo solo se atendían unas 35. Al revisar la métrica de concurrencia en CloudWatch vimos la causa:

```
Metricas de la Lambda durante la prueba ANTES de la mejora (CloudWatch, ventanas de 10 min):
Duracion promedio (ms): 51.9 / 48.6
Duracion maxima (ms): 293 / 3543 (arranques en frio)
Concurrencia maxima: 10 / 10  <- limite de la cuenta nueva
Throttles (rechazadas por limite): 8070 / 8937
Invocaciones: 4034 / 5280
Errores de codigo: 0 / 0
```

Figura 4. *Métrica de concurrencia de la Lambda en CloudWatch durante la prueba, ANTES de la mejora: la concurrencia máxima se quedó en 10, el límite de la cuenta.*

1. La cuenta nueva solo permite **10 Lambdas ejecutándose al mismo tiempo**, y cada `GET /top` recorría toda la tabla (`scan`) en unos 50 ms.
2. Además, el límite que habíamos puesto en API Gateway (100 peticiones por segundo) rechazaba otra parte con error 429.

Hicimos dos mejoras: una **caché del top de 2 segundos** en cada Lambda (un top con 2 segundos de retraso está bien para una tabla de líderes) y subimos el límite de API Gateway a 1000 peticiones por segundo. Repetimos la misma prueba:

```
Metricas de la Lambda durante la prueba DESPUES de la mejora (cache de /top 2 s; API Gateway 1000 req/s):
Duracion promedio (ms): 13.0 (antes 50)
Duracion p50 (ms): 2.1   p90: 48.3   minima: 0.99
Concurrencia maxima: 10  <- sigue siendo el limite de la cuenta nueva
Throttles: 98376
Invocaciones atendidas: 29695 (antes 9314)

Resumen Locust (500 usuarios, 5 min):
ANTES : 132673 peticiones, 10503 exitosas (~35/s), errores 429 (API Gateway) y 503 (Lambda)
DESPUES: 133367 peticiones, 29786 exitosas (~99/s), solo 503 (limite de 10 Lambdas a la vez)
Conclusion: la cache y el nuevo limite eliminaron los 429 y triplicaron lo atendido; el cuello de botella rest
ante es la cuota de concurrencia de Lambda (10) de la cuenta nueva. Solucion: solicitar aumento de cuota en Se
rvice Quotas (valor normal de AWS: 1000).
```

Figura 5. *Las mismas métricas DESPUÉS de la mejora, con el resumen de Locust.*

	Antes	**Después**
Peticiones atendidas con éxito	~35 por segundo	**~99 por segundo**
Errores 429 (API Gateway)	102,607	**0**
Duración promedio de la Lambda	50 ms	**13 ms**
Latencia p50 / p95 / p99	75 / 130 / 260 ms	77 / 110 / 210 ms

Lo atendido se triplicó. El límite que queda es la cuota de 10 Lambdas de la cuenta nueva; lo normal en AWS es 1000 y se pide en *Service Quotas*.

5. Costo: serverless contra una VM

Con los precios de lista de AWS (9 de noviembre), para el escenario del material de 5 millones de consultas al mes:

Concepto	**Cálculo**	**USD al mes**
Lambda, peticiones	(5 M − 1 M gratis) × 0.20 por millón	0.80
Lambda, cómputo	16,250 GB-s, dentro de los 400,000 gratis	0.00
API Gateway	5 M × 1.00 por millón	4.00 – 5.00
DynamoDB, escrituras	1 M × 0.625 por millón	0.63
DynamoDB, lecturas	con la caché, casi ninguna	< 1
Total serverless		**≈ 6 – 7**
VM pequeña encendida 24/7	máquina, IP pública y disco	≈ 15 – 20

Además, sin jugadores la opción serverless cuesta casi cero, mientras que la VM cuesta lo mismo aunque nadie juegue.

Conclusión

La tabla de líderes funcionó desde el primer día sin administrar ningún servidor, y es más barata que una VM para este uso. La prueba de carga fue lo más valioso: sin ella no habríamos descubierto el límite de 10 Lambdas ni que recorrer toda la tabla en cada consulta era el problema. Con una caché sencilla triplicamos la capacidad. Para producción pediríamos la cuota de concurrencia y cambiaríamos el `scan` por un índice ordenado por puntos.